Chapter 2: Data Representation and Boolean Algebra

Number System:-

A Number System is a method of representing numbers using a set of digits. The total number of these digits is known as base or Radix.

Types of Number Systems

1. Decimal Number System:

It is the number system that we commonly use in our daily life. Its base is 10 with digit 0 to 9. The digit with the highest place value is the Most Significant Digit (MSD). The digit with the lowest place value is the Least Significant Digit (LSD).
e.g., 679.85 (MSD is 6 & LSD is 5)

2. Binary number system:-

It is the number system that is used in computer for representing ON (1) and OFF (0). Its base is 2 with digits 0 and 1. In binary number, the left most bit is known as Most Significant Bit (MSB). The right most bit is known as Least Significant Bit (LSB).
e.g. 1101

3. Hexa decimal number system:-

It is a compact way to represent binary numbers as each hexa decimal digit corresponds to four binary digits. Its base is 16 with digits 0-9 and A – F.
e.g., 2AB

Comparison of Number Systems:

Number System	Base	Digits Used
Decimal	10	0–9
Binary	2	0,1
Octal	8	0–7
Hexadecimal	16	0–9, A, B, C, D, E, F

Decimal to Binary conversion: -

1. Divide the decimal number repeatedly by 2 until the quotient becomes 0
2. Note down the remainders.
3. Write the remainders in reverse order to get the binary number.

Decimal to Hexadecimal conversion: -

1. Divide the decimal number repeatedly by 16 until the quotient becomes 0
2. Note down the remainders and represent with its equivalent hexadecimal digit.
3. Write the hexadecimal digits in reverse order to get the hexadecimal number.

Binary to Decimal conversion: -

1. Multiply binary digit by power of 2 and find their sum.
2. Binary to Hexadecimal conversion: -
3. Convert to hexadecimal equivalent for each 4 bits group from right to left.

Binary to Hexadecimal conversion: -

1. Divide the binary number into groups of 4 bits, starting from the right.
2. Replace each 4-bit group with its corresponding hexadecimal symbol.
3. Combine all hexadecimal symbols to get the final number.

Hexadecimal to Binary Conversion: -

- Converting each hexadecimal digit to its 4 bit binary equivalent

Binary Addition

The rules for adding two binary numbers are

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Binary Subtraction

The rules for subtracting binary numbers are

A	B	Difference	Borrow
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

Data Representation:-

In computer all types of data such as numbers, text, audio and video are represented in binary form using 0s and 1s.

Representation of Integers:-

The three methods of representing integers are Sign and magnitude representation, 1's complement representation and 2's complement representation.

- **Sign and magnitude representation**:- In this method MSB (Left most bit) is used for representing sign of integer and remaining 7 bits are used for representing magnitude of integer. For negative numbers sign bit is 1 and for positive numbers sign bit is 0.
  e.g., -3 and be represented as

1	0	0	0	0	0	1	1

  Sign Magnitude

- **1's complement representation**:- In this method, negative numbers are stored in its 1's complement form and positive numbers are stored in its original binary form. 1's complement of a binary number can be obtained by replacing every 0 with 1 and 1 with 0.
  e.g., -3 can be represented as 11111100 (Binary for 3 in 8 bit form is 00000011, hence
  -3 is represented in its 1's complement as 11111100)

- **2's complement representation:-** In this method, negative numbers are stored in its 2's complement form and positive numbers are stored in its original binary form. 2's complement of a binary number can be obtained by adding 1 to its 1's complement.
  e.g., -3 can be represented as 11111101 (Binary for 3 in 8 bit form is 00000011, its 1's complement is 11111100, hence the 2's complement of -3 is 11111101)

Subtraction using 1's complements:- The steps for subtraction are

1. Add 0's to the left of subtrahend (number to be subtracted) to make minuend (number from which subtracting) and subtrahend with same number of bits
2. Find 1's complement of subtrahend.
3. Add it with minuend.
4. If there is carry bit, add that carry also to get the answer.

e.g.:- subtract 100 from 1010

minuend	= 1010
subtrahend	= 0100
1's complement of subtrahend	= 1011
Add it with minuend	= 1010 +
	1011
	10101 (Carry bit is added with result)
Result is	0101+1 = 0110

Subtraction using 2's complements:- The steps for subtraction are

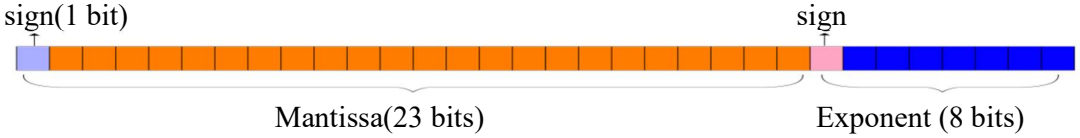
1. Add 0's to the left of subtrahend (number to be subtracted) to make minuend (number from which subtracting) and subtrahend with same number of bits
2. Find 2's complement of subtrahend.
3. Add it with minuend.
4. If there is carry bit, ignore it.

e.g.:- subtract 100 from 1010

minuend	= 1010
subtrahend	= 0100
2's complement of subtrahend i.e., 0100	= 1100
Add it with minuend	= 1010 +
	1100
(ignore this 1)	10110
Result is	0110

Representation of floating point numbers:- A floating point number consists of an integer part (mantissa) and fractional (exponent) part. In computer memory a floating point number is divided into 3 parts – sign bit, exponent and mantissa. Floating point numbers can be defined in two formats
Single precision (32 bits):- 1 bit for sign, 8 bits for exponent, 23 bits for mantissa.
Double precision (64 bits):- 1 bit for sign, 11 bits for exponent and 52 bits for mantissa.

For eg., 25.45 can be written as 0.2545×10^2 . Here 0.2545 is the mantissa and 2 is the exponent



Representation of Characters

ASCII:-

ASCII stands for American Standard Code for Information Interchange. It uses 7 bits to represent each character in computer memory. 128 characters can be represented by ASCII code. Another version is ASCII-8 called extended ASCII which uses 8 bits.

Eg., ASCII code of A is 65 (its binary equivalent is 1000001)

Unicode:-

Unicode can represent data in almost all written languages of the world. The Unicode assigns a unique number called a code point to every character. Unicode uses 16 bits or more than 16 bits to represent over 1,00,000 characters. Unicode is the global standard for text representation.

Representation of Image, Audio and Video:-

1. Image Representation:-

In computer, images are stored in digital format. Image is made up of a large number of tiny dots called pixels (picture element). The clarity of an image depends on its resolution (the number of pixel in an image) and colour depth (the no. of bits used to represent the colour of each pixel). Digital images are classified as

- Bitmap (Raster) images:- made up of pixels arranged in rows and columns. E.g., photographs.
- Vector images:- made up of lines and curves. E.g., diagrams

Different file formats used to store digital images are BMP (Bitmap), JPEG (Joint Photographic Experts Group), GIF (Graphic Interchange Format) etc.

2. Audio Representation:-

To store sound in a computer, it first converted from analog to digital form (digitisation). The number of bits used to store each sample for sound level is called the bit depth. Various file formats are used to store digital sound such as WAV, AIFF, MP3 etc

3. Video Representation:-

A video is a collection of moving images. The quality of a video depends on

- Frame rate (no. of frames displayed per second)
- Resolution (no. of pixel)
- Bit rate (amount of data used to store one second of video).

Some video formats are AVI, MP4, MOV etc

Boolean Algebra

Boolean Algebra works with two values 0 and 1 representing False and True. Boolean Algebra is used in designing digital circuits, switching systems, computer hardware etc.

Boolean Operators and Logic Gates:-

Boolean Algebra uses three fundamental logical operations AND, OR, NOT. A logic gate is an electronic circuit for implementing logic operations.

(a) OR operation and OR Gate:- The OR operator performs logical addition and the symbol used for this operation is + (plus). The truth table and the symbol for OR gate is

A	B	A +B
0	0	0
0	1	1
1	0	1
1	1	1



(b) AND Operation and AND Gate:- The AND operator performs logical multiplication and the symbol used for this operation is . (dot). The truth table and the symbol for AND gate is

A	B	A .B
0	0	0
0	1	0
1	0	0
1	1	1



(c) NOT Operation and NOT Gate: - The NOT operator performs logical negation and the symbol used for this operation is – (over bar). It is a unary operator and requires only one operand.

A	$\overline{A}$
0	1
1	0



Principle of duality:- Each Boolean expression has a dual form which can be derived by

1. Changing each OR(+) to AND(.)
2. Changing each AND(.) to OR(+)
3. Changing each 0 to 1 and 1 to 0

Boolean Identities

Identity / Law	Statement 1	Statement 2
Idempotent	A + A = A	A . A = A
Commutative	A + B = B + A	A . B = B . A
Associative	A + (B + C) = (A + B) + C	A . (B . C) = (A . B) . C
Distributive	A . (B + C) = A . B + A . C	A + (B . C) = (A + B) . (A + C)
Absorption	A + (A . B) = A	A . (A + B) = A
Involution	$\overline{\overline{A}} = A$	

De Morgan's theorems:- De Morgan's theorems can be stated as

(1) The complement of sum of Boolean variables is equal to product of their individual complements

(2) The complement of product of Boolean variables is equal to sum of their individual complements., i.e.,

i. $\overline{A + B} = \overline{A} \cdot \overline{B}$

ii. $\overline{A \cdot B} = \overline{A} + \overline{B}$

De Morgan's first theorem

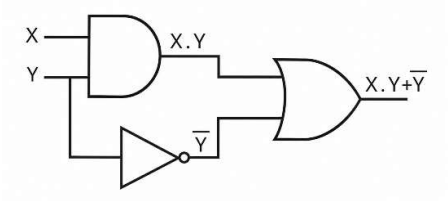
A	B	$\overline{A}$	$\overline{B}$	A+B	$\overline{A + B}$	$\overline{A} \cdot \overline{B}$
0	0	1	1	0	1	1
0	1	1	0	1	0	0
1	0	0	1	1	0	0
1	1	0	0	1	0	0

De Morgan's second theorem

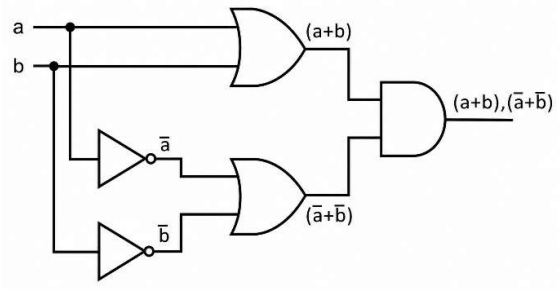
A	B	$\overline{A}$	$\overline{B}$	A.B	$\overline{A \cdot B}$	$\overline{A} + \overline{B}$
0	0	1	1	0	1	1
0	1	1	0	0	1	1
1	0	0	1	0	1	1
1	1	0	0	1	0	0

Circuit Designing using AND, OR and NOT gates

1) Construct the logic circuit for the Boolean expression: $\overline{X} \cdot Y + Y$



2) Construct the logic circuit for the Boolean expression: $(\overline{a} + \overline{b}) \cdot (a + b)$



Universal gates:- The NAND and NOR gates are called universal gates because any other logic gate (AND, OR, NOT) can be constructed using only NAND or only NOR gates.

1) **NAND gate**:- The NAND gate is a combination of AND gate followed by a NOT gate. The symbol for NAND gate is



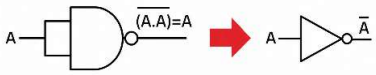
2) **NOR gate:-** The NOR gate is a combination of OR gate followed by a NOT gate. The symbol for NOR gate is



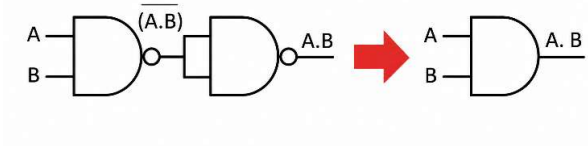
Implementation of basic gates using NAND:-

We can design all basic gates (AND, OR and NOT) using only NAND gates.

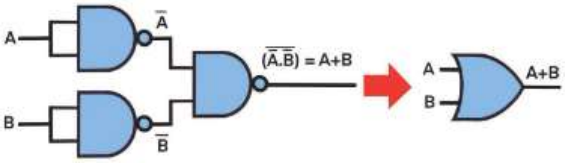
1) NOT gate using NAND gate:- We can implement a NOT gate by applying the same signal to both inputs of a NAND gate.



2) AND gate using NAND gate:- We can implement an AND gate by using a NAND gate followed by another NAND gate.



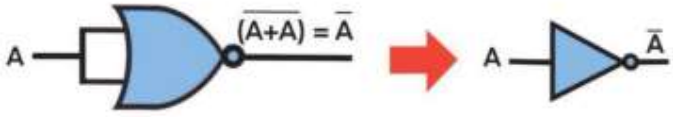
3) OR gate using NAND gate:- We can implement an OR gate by using three NAND gates.



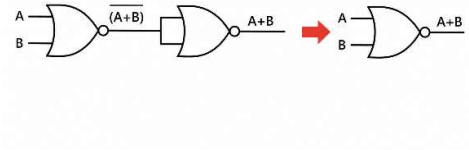
Implementation of basic gates using NOR :-

We can design all basic gates (AND, OR and NOT) using only NOR gates.

1) NOT gate using NOR gate:- We can implement a NOT gate by applying the same signal to both inputs of a NOR gate.



3) OR gate using NOR gate:- We can implement an OR gate by using a NOR gate followed by another NOR gate.



AND gate using NOR gate:- We can implement an AND gate by using three NOR gates.

